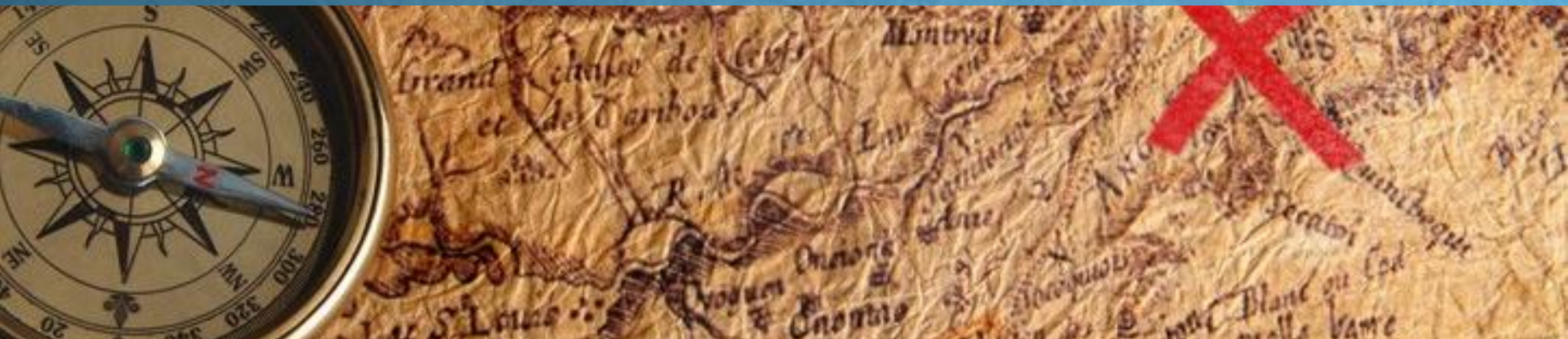


5. Parametrización Aprendizaje supervisado



MD005 – Sistemas Basados en el Conocimiento

Jordina Torrents - jordina.torrents@ext.salle.url.edu



¿Preguntas ejercicios?

¿Qué hemos hecho hasta ahora con los algoritmos?

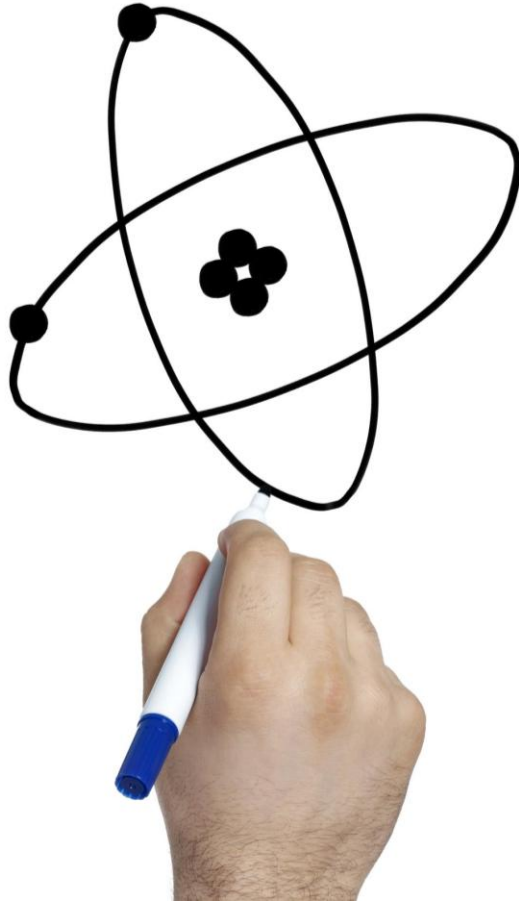
Directamente hemos utilizado la configuración vanilla (por defecto) o hemos jugado con unos pocos parámetros (sobre todo en la clasificación).

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini', splitter='best',  
max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0,  
max_features=None, random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0,  
class_weight=None, ccp_alpha=0.0, monotonic_cst=None)
```

¿Qué hemos hecho hasta ahora con los algoritmos?

- **Criterion:** Medida de la pureza en los nodos, 3 opciones posibles (gini, entropy y log_loss)
- **Splitter:** Estrategia para dividir, 2 opciones (best, random)
- **Max_depth:** Profundidad máxima, si no se define dividirá hasta que las hojas tengan Pureza completa o la limitación de otros parámetros (min_samples_split)
- **Min_samples_split:** Número mínimo de muestras en un nodo para dividir.
- **Min_samples_leaf:** Muestras mínimas en cada hoja final.
- **Max_features:** Cantidad de variables a tener en cuenta cada vez que se plantea expandir el nodo.
- **Max_leaf_nodes:** Control del hojas finales.
- **Min_impurity_decrease:** límite sobre el que se mide la Pureza, si no mejora más de lo estipulado no dividiremos.
- **Class_weight:** balance de los pesos en función de la clase, para problemas multi-clase en los que no hay un balance en los datos.
- **Ccp_alpha:** parámetro de complejidad para realizar el prunado del árbol, reducir su tamaño.

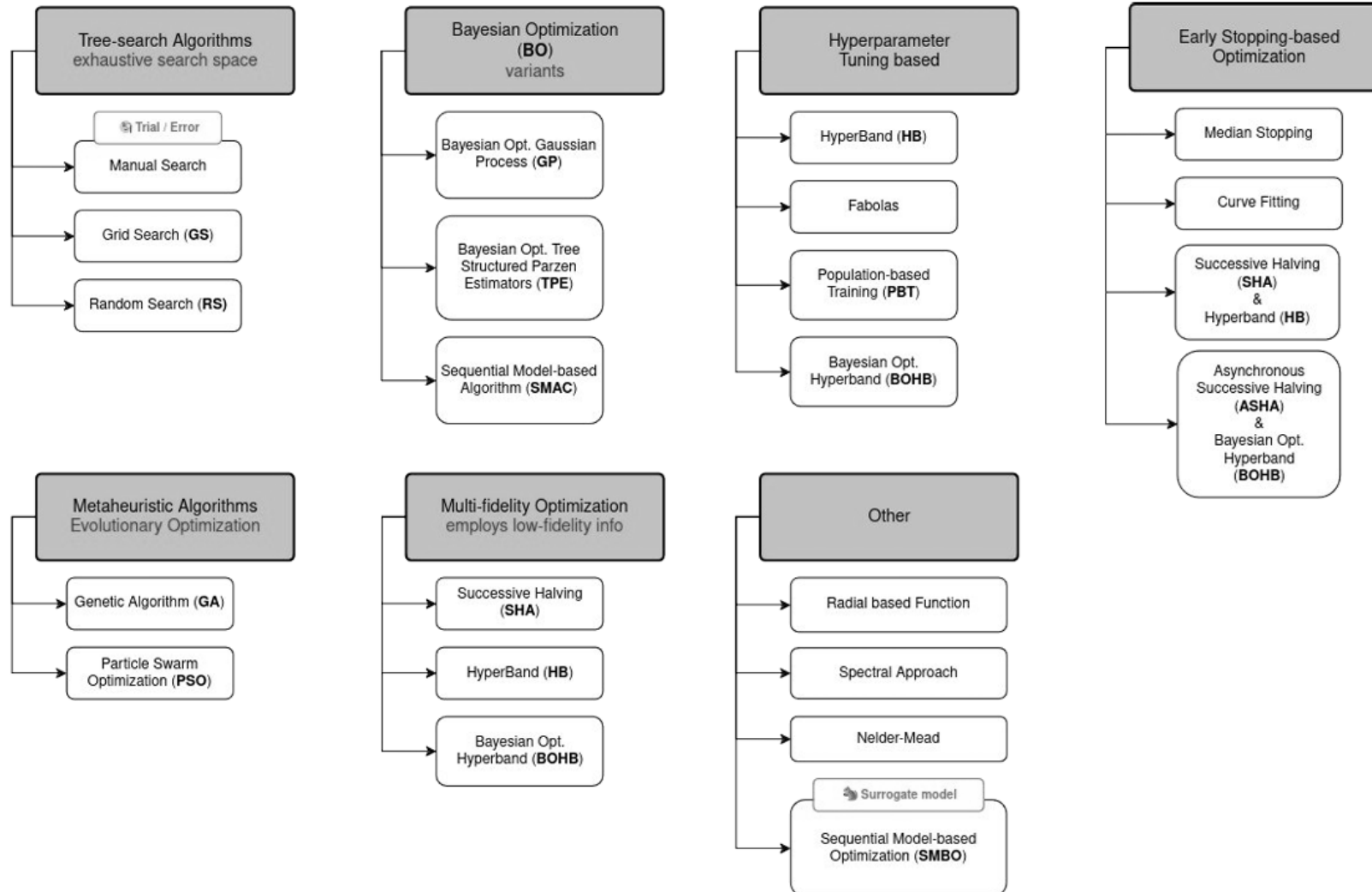
Ajuste de Modelos: Optimización de Hiper-parámetros

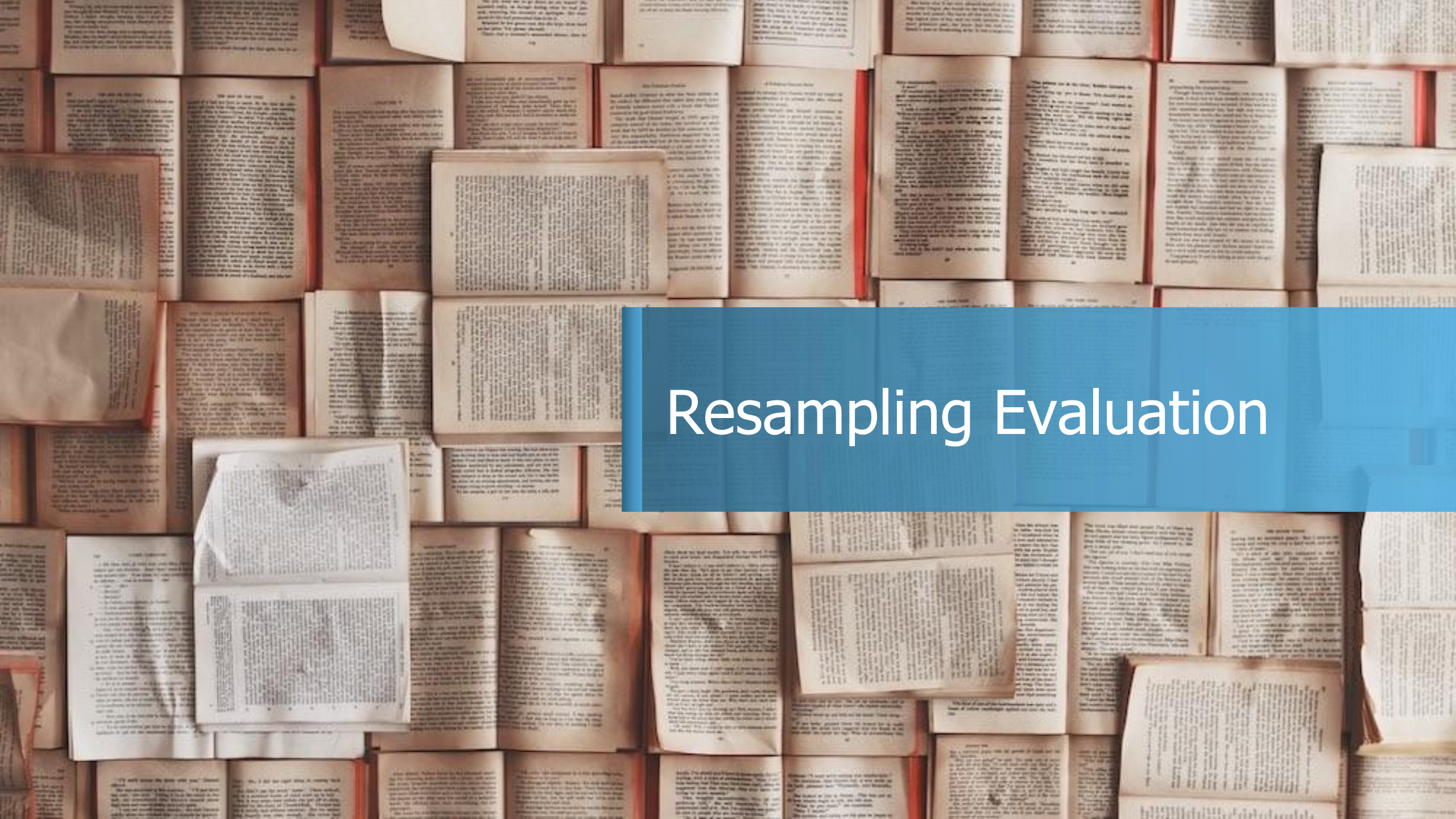


- No existe forma de conocer de antemano los valores adecuados.
- El model tuning es el proceso que identifica los hiperparámetros óptimos de un modelo.
 - Se exploran diversas combinaciones para mejorar la efectividad.
 - El uso de técnicas de resampling (cross-validation o bootstrapping) valida el rendimiento del modelo de forma **ROBUSTA**.
 - El objetivo es minimizar el sobreajuste y mejorar la generalización.

Este enfoque iterativo asegura un rendimiento óptimo bajo ciertas restricciones.

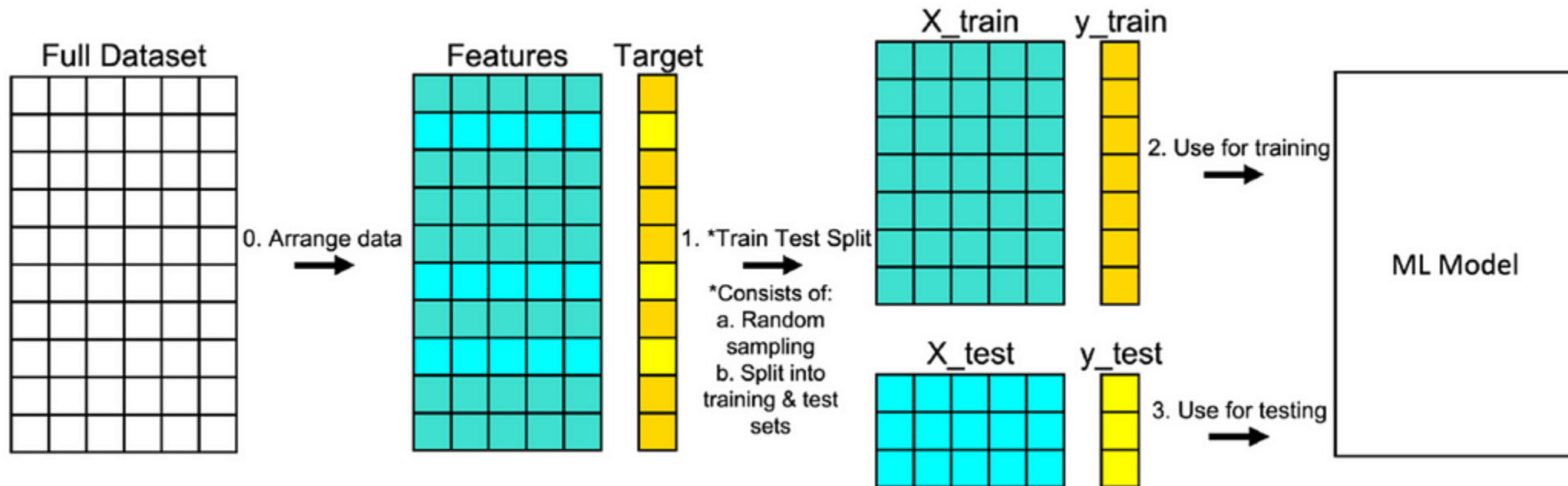
Algoritmos



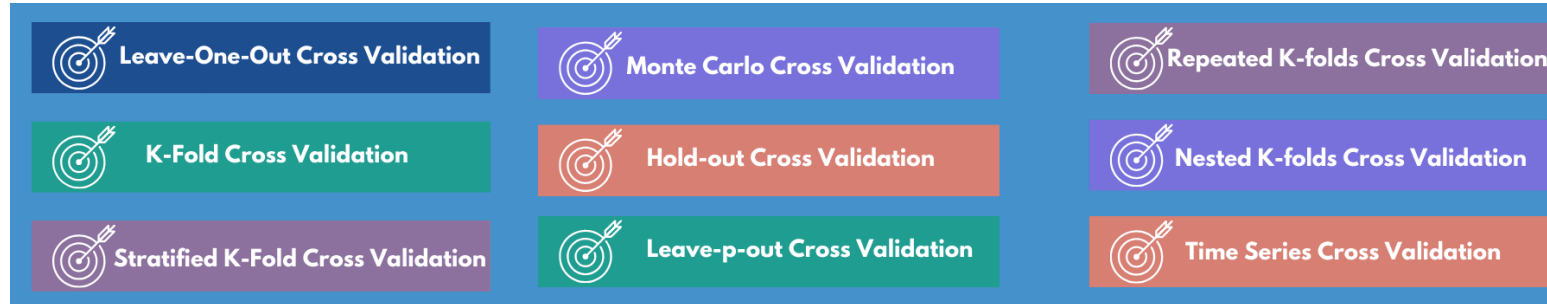


Resampling Evaluation

Hasta ahora...



Tipos de Cross-Validation



Hold-Out:

- Divide el dataset una sola vez (train / test).
- Muy rápido, pero **alta varianza**.
- Sensible a cómo se hace la partición.
- Útil como baseline o con datasets grandes.

K-Fold Cross-Validation:

- Divide los datos en K folds.
- Cada fold se usa una vez como test.
- Buen equilibrio entre bias y varianza.
- Estándar en muchos problemas.

Leave-One-Out CV (LOOCV):

- Caso extremo de K-Fold con $K = N$.
- 1 muestra para test, $N-1$ para train.
- Bias bajo, **varianza alta**.
- Coste computacional elevado.

Leave-P-Out CV:

- Generalización de LOOCV.
- P muestras se dejan fuera en cada iteración.
- Muy costoso computacionalmente.
- Poco usado en la práctica.

Monte Carlo Cross-Validation:

- Repetidas divisiones aleatorias train/test.
- El tamaño de train/test es fijo.
- Más flexible que K-Fold.
- También llamado *Repeated Random Subsampling*.

Repeated K-Fold:

- K-Fold repetido varias veces con distintas particiones.
- Reduce la varianza de la estimación.
- Más robusto, pero más caro computacionalmente.
- Útil cuando el dataset es pequeño o inestable.

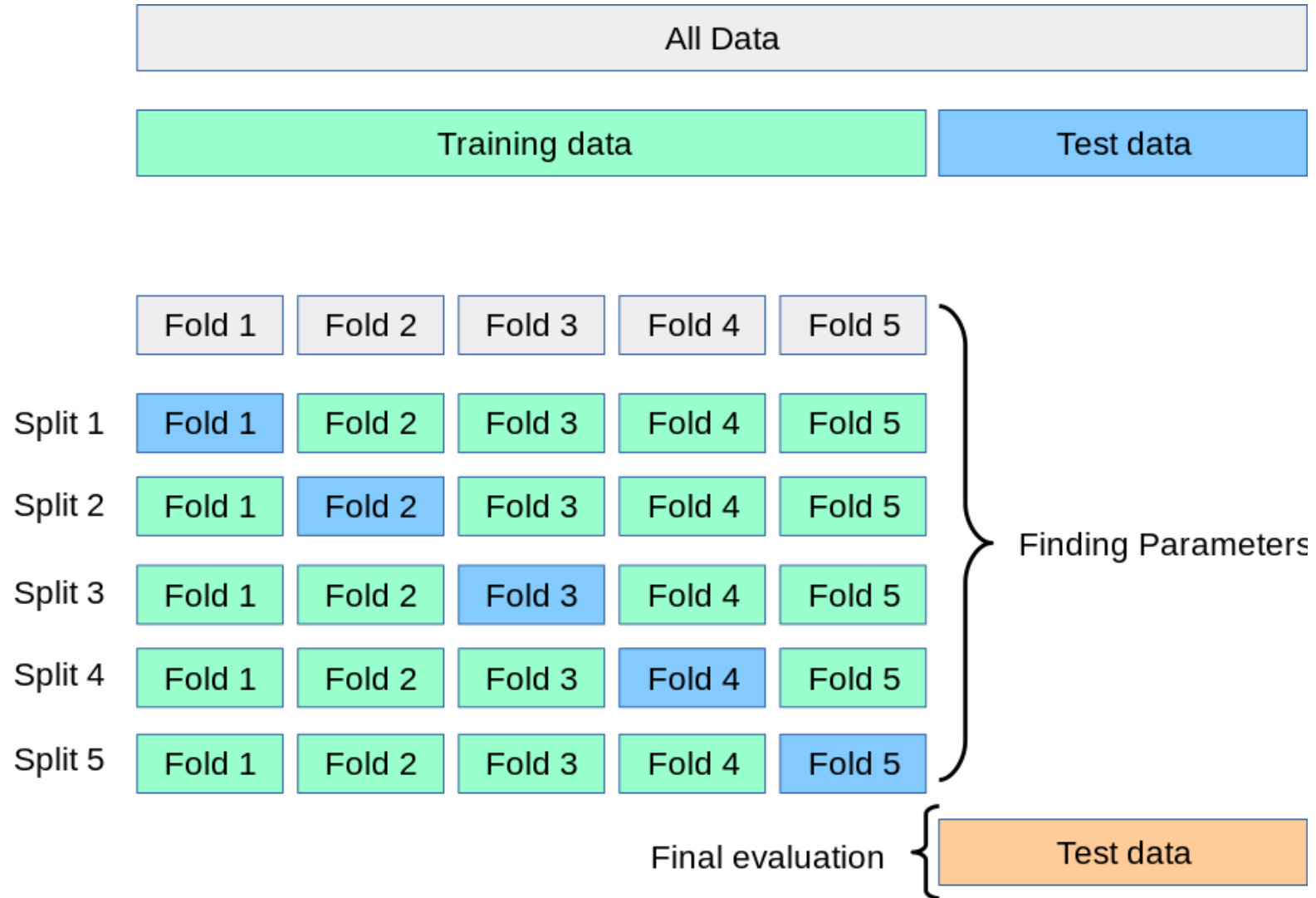
Nested K-Fold:

- Cross-validation dentro de cross-validation.
- CV interno: selección de hiperparámetros.
- CV externo: estimación del rendimiento.
- Evita **data leakage** y optimismo en la evaluación.

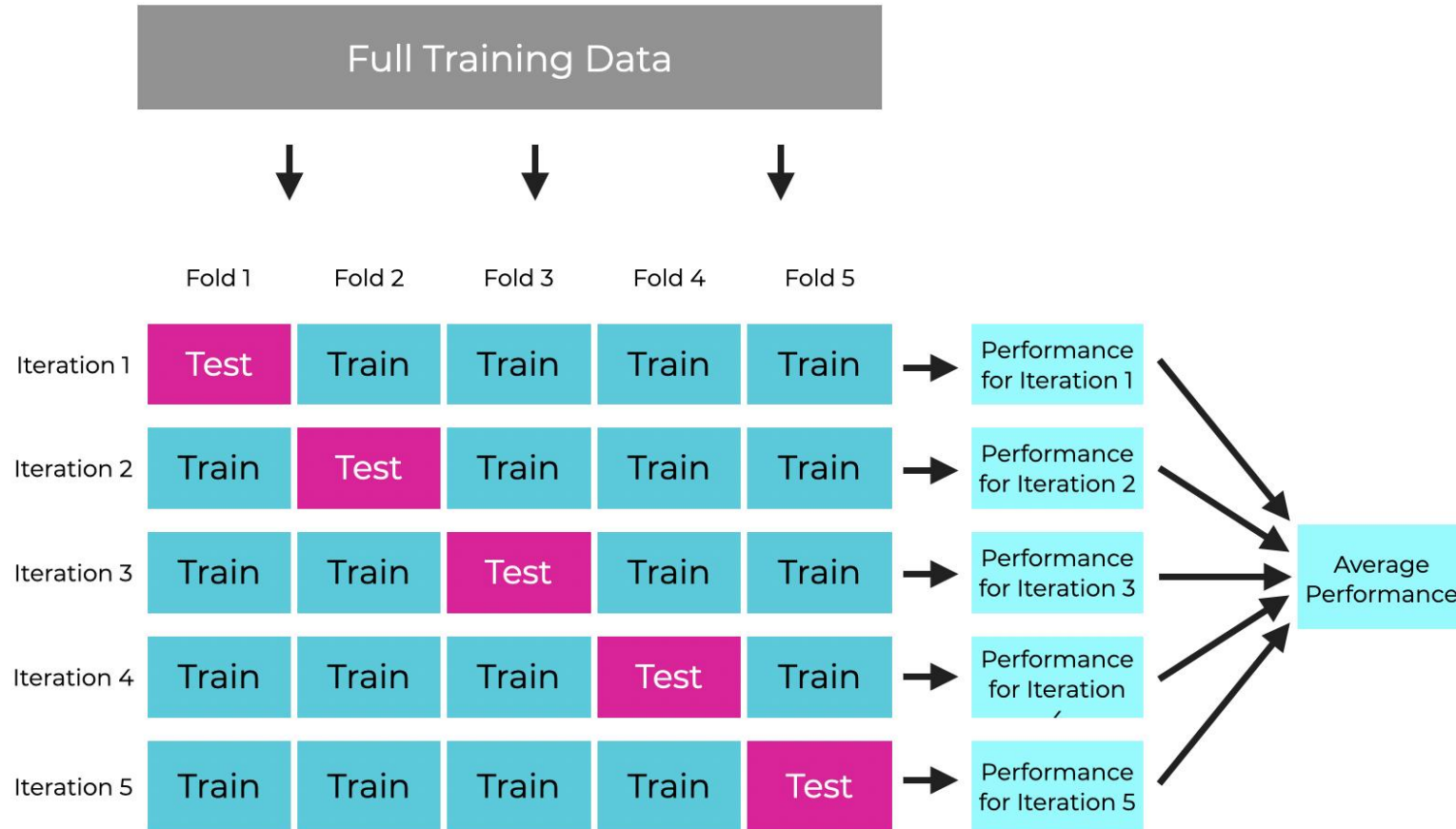
Time Series Cross-Validation:

- Respeta el orden temporal de los datos.
- Entrena con el pasado, evalúa en el futuro.
- Usa ventanas crecientes o deslizantes.
- **Obligatorio** en series temporales.

(Stratified) Cross-Fold Validation



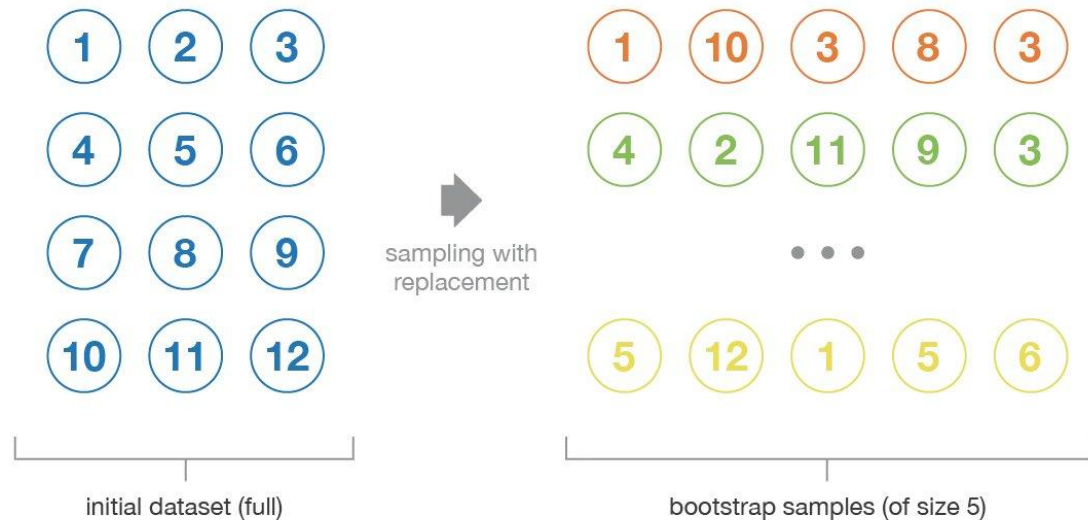
(Stratified) Cross-Fold Validation



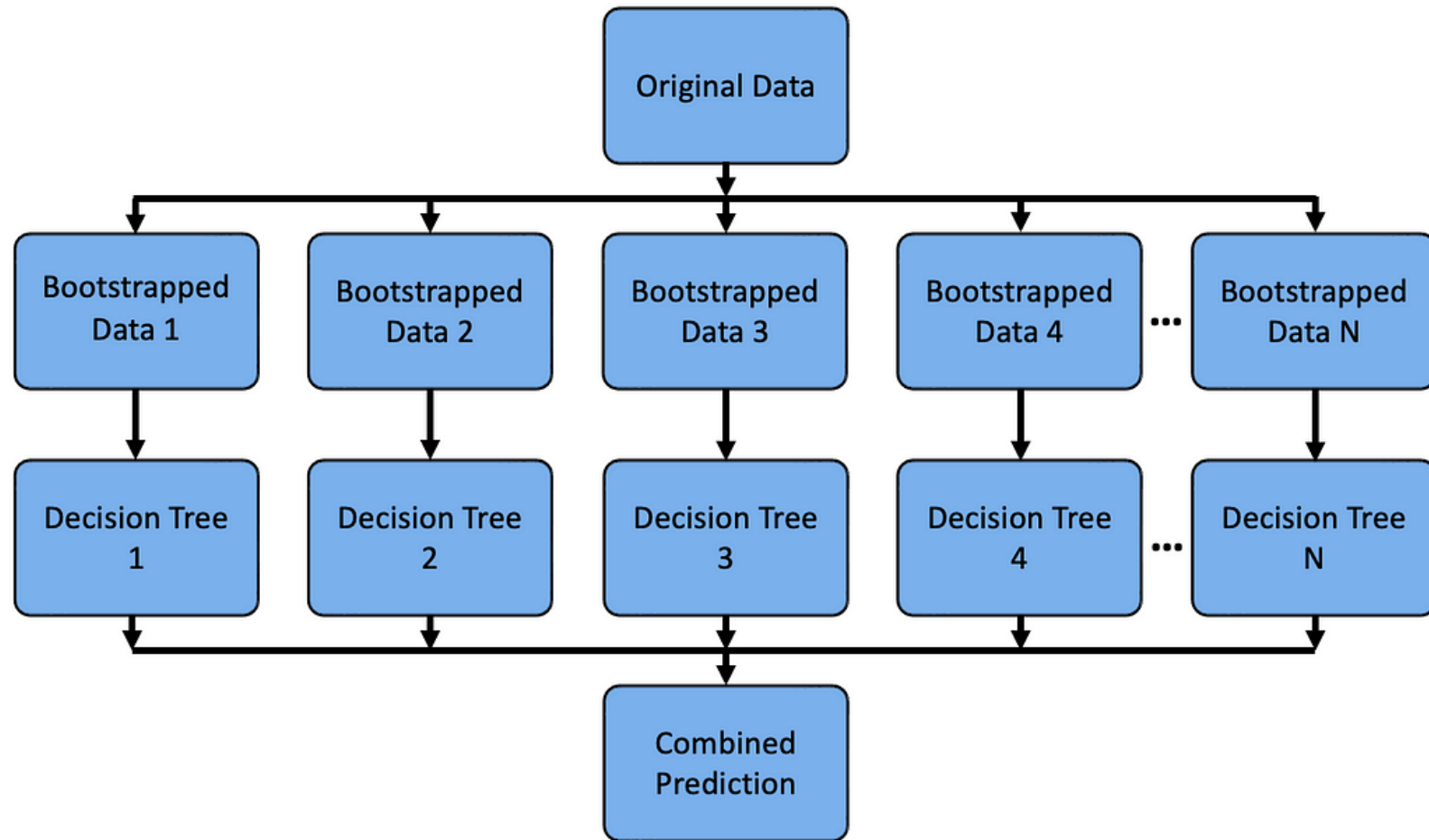
Problema: Perdemos representatividad en las iteraciones

Bootstrapping

- ❑ En la práctica casi nunca tendremos acceso a múltiples muestras independientes. Con **bootstrapping** podemos **simular** la obtención de varias muestras a partir de un *dataset*, llamadas muestras *bootstrap*
- ❑ **Concepto:** Dado un **dataset de tamaño N** , se obtienen **muestras de tamaño B** al extraer B observaciones al azar permitiendo que una misma observación sea elegida más de una vez (muestreo con reemplazamiento).
- ❑ **Suposiciones:**
 - **Representatividad:** N debe ser lo suficientemente grande para capturar la complejidad de la distribución subyacente, garantizando así que muestrear el *dataset* es aproximadamente igual a muestrear la verdadera distribución.
 - **Independencia:** N debe ser suficientemente grande en comparación con el tamaño B de las muestras extraídas para evitar una correlación excesiva entre dichas muestras



Bootstrapping

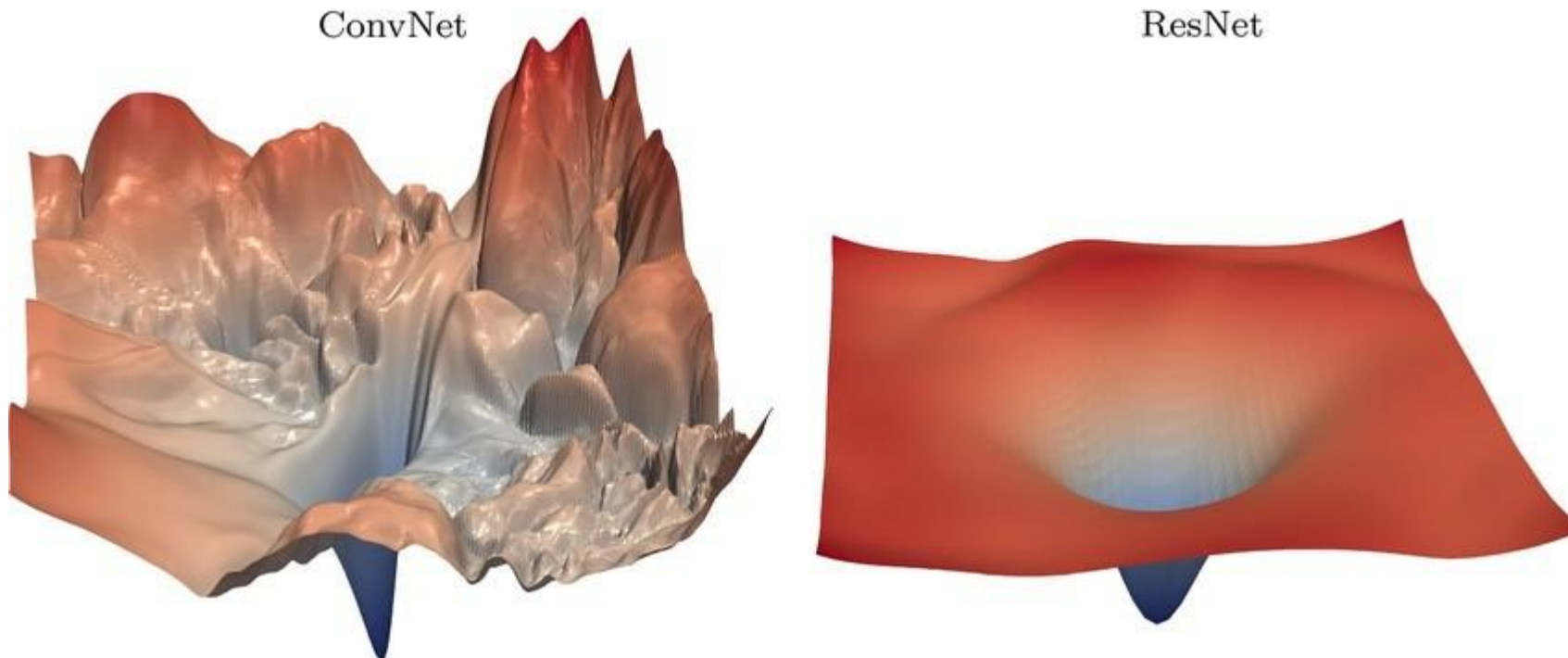




Exhaustive Search Space

¿Cómo vamos a encontrar valores?

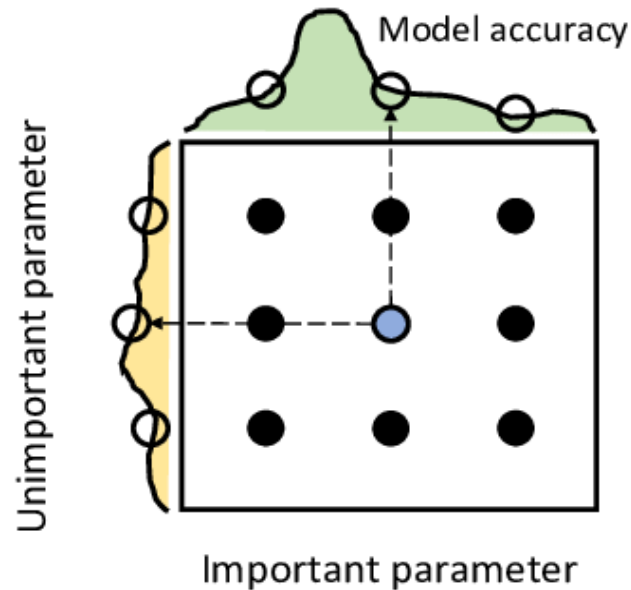
Hay varios métodos que nos permiten mejorar los resultados de nuestro algoritmo simplemente evaluando combinaciones de éstos y encontrando mínimos locales mucho mejores que a simple prueba y error.



Método 1: Grid Search

Método más intuitivo para encontrar mejores parametrizaciones, consiste en probar a fuerza bruta un conjunto de parametrizaciones y nos quedaremos con el que mejor resultados proporcione.

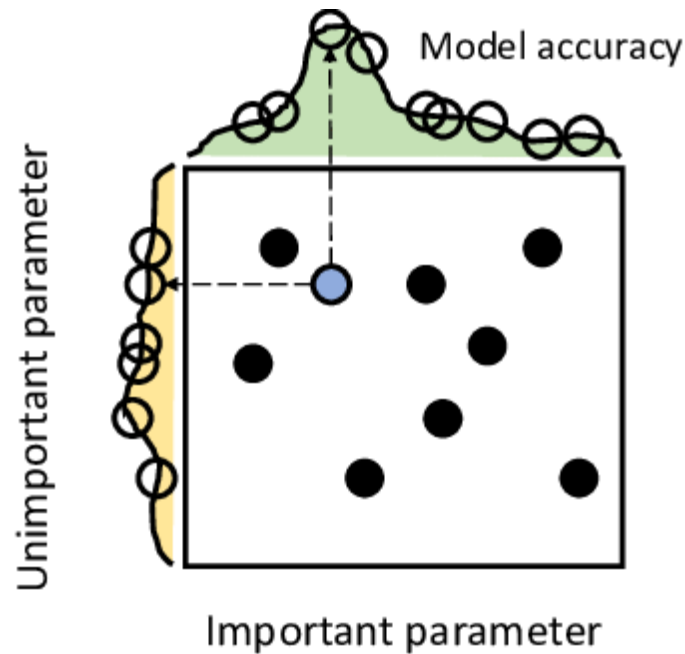
¡Cuidado con la matriz de parámetros!



- Alto consumo de recursos.
- Mala escalabilidad.
- No garantiza encontrar un resultado óptimo, sólo el mejor de la combinación que se proporciona.

Método 2: Random Search

Similar al anterior, en este caso no se comprueban todos, solamente casos seleccionados de forma aleatoria.



- Menos recursos que RandomSearch al no comprobar todas las combinaciones.
- Dependemos de la aleatoriedad.
- Necesitaremos aplicar **cross validation** para asegurar los resultados, al no ser determinista a la hora de escoger los parámetros.
- No garantiza encontrar un resultado óptimo, pero si el mínimo global se encuentra dentro de los valores definidos, con > 60 iteraciones debería encontrar una combinación muy próxima.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

Método 3: Population-based Search

A diferencia de métodos como Grid Search o Random Search, que prueban puntos individuales, **population-based search explora muchas soluciones al mismo tiempo**. Cada solución en la población se evalúa usando una función *fitness*. Se generan nuevas soluciones combinando, mutando o seleccionando las mejores de la población anterior.

1. Genetic Algorithms (GA)

- La población = individuos (candidatos).
- Se seleccionan los mejores, se cruzan (crossover) y mutan para formar la siguiente generación.

2. Particle Swarm Optimization

- La población = partículas que representan soluciones.
- Cada partícula se mueve en el espacio de búsqueda influida por su mejor posición (y global).

3. Differential Evolution

- Similar a GA, pero usa operaciones de diferencia entre vectores para generar nuevos individuos.

4. Estimation of Distribution Algorithms

- Se ajusta un modelo probabilístico de la población y se muestrean nuevas soluciones.



Evolutionary Optimization

Algoritmos genéticos

Los **Algoritmos Genéticos** son algoritmos de optimización búsqueda y aprendizaje inspirados en los procesos de “Evolución Natural” y “Evolución Genética”.

Trabajan con una **población de individuos**, cada uno de los cuales **representa una solución factible a determinado problema**. A cada individuo se le asignan un valor o puntuación relacionada con la **función de adaptación** (*fitness*).

Se utiliza operadores genéticos **probabilísticos de selección, recombinación y mutación**.

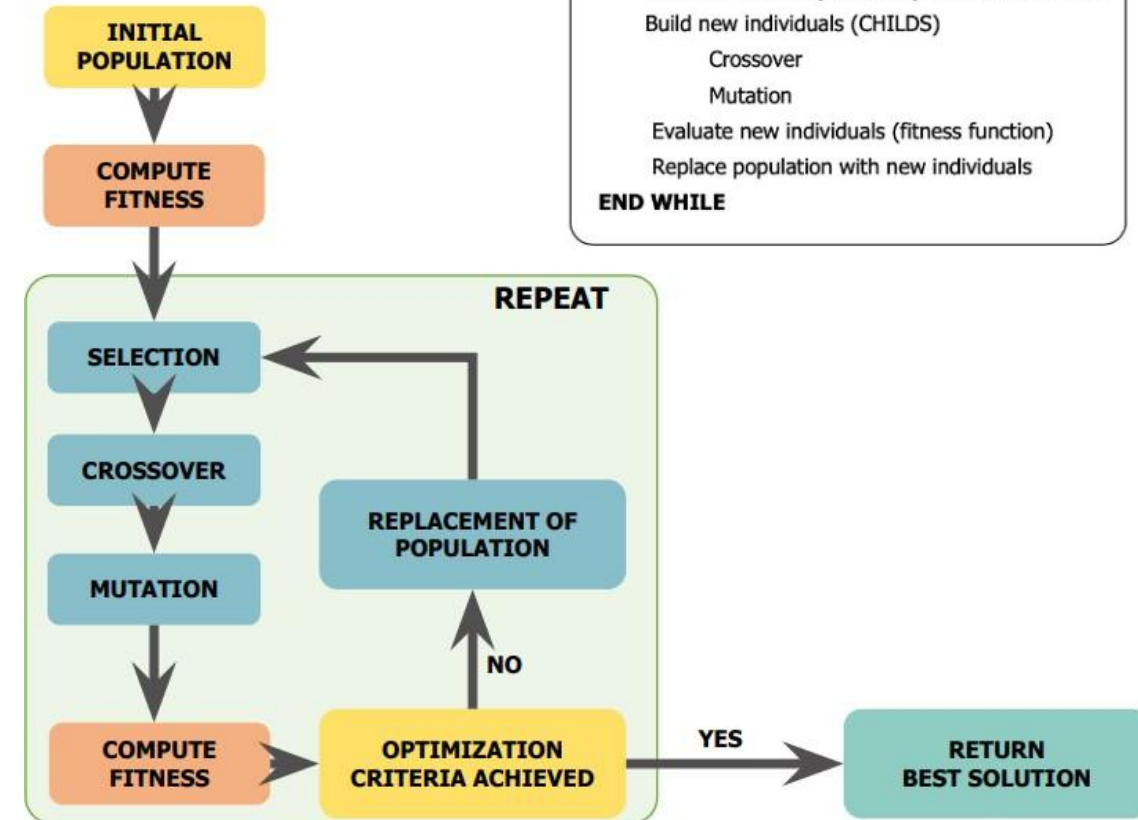
VENTAJAS

- Simplicidad conceptual del mecanismo de exploración.
- Superiores a métodos tradicionales.
- Pueden incorporar conocimiento sobre el dominio del problema.
- Pueden hibridarse con otras técnicas de búsqueda.
- Son robustas a los cambios dinámicos.
- En general, pueden auto-adaptar sus parámetros.
- Explotan de modo natural las ventajas del procesamiento paralelo.

Algoritmos genéticos

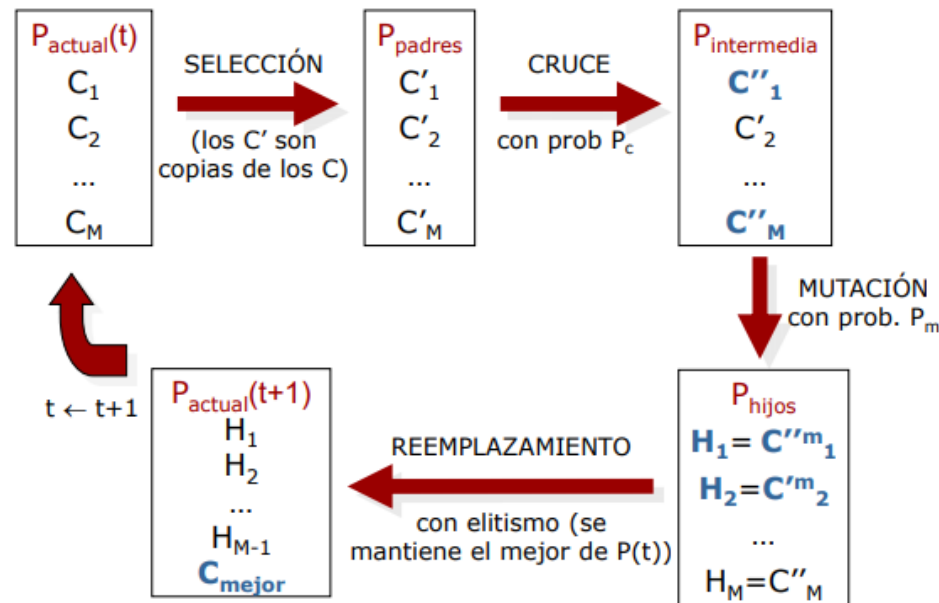
Basados en la selección natural, escogen la mejor combinación a partir de cruzar combinaciones anteriores y permitiendo la mutación.

Para seleccionar qué combinación (individuos) serán los que sobrevivirán, se utiliza una función de evaluación que comprueba qué tan bien se ajusta al objetivo.

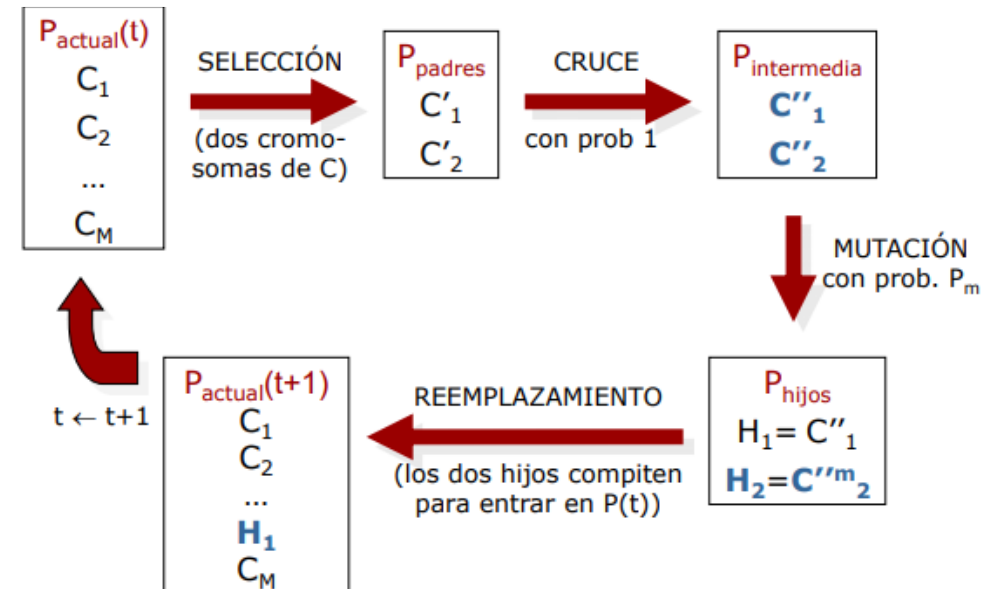


Modelo Generacional i Estacionario

Modelo generacional: Durante cada iteración se crea una población completa con nuevos individuos. La nueva población reemplaza directamente a la antigua.



Modelo estacionario: Durante cada iteración se escogen dos padres de la población (diferentes muestreos) y se les aplican los operadores genéticos. El/los descendiente/s reemplazan a uno/dos cromosoma/s de la población inicial.



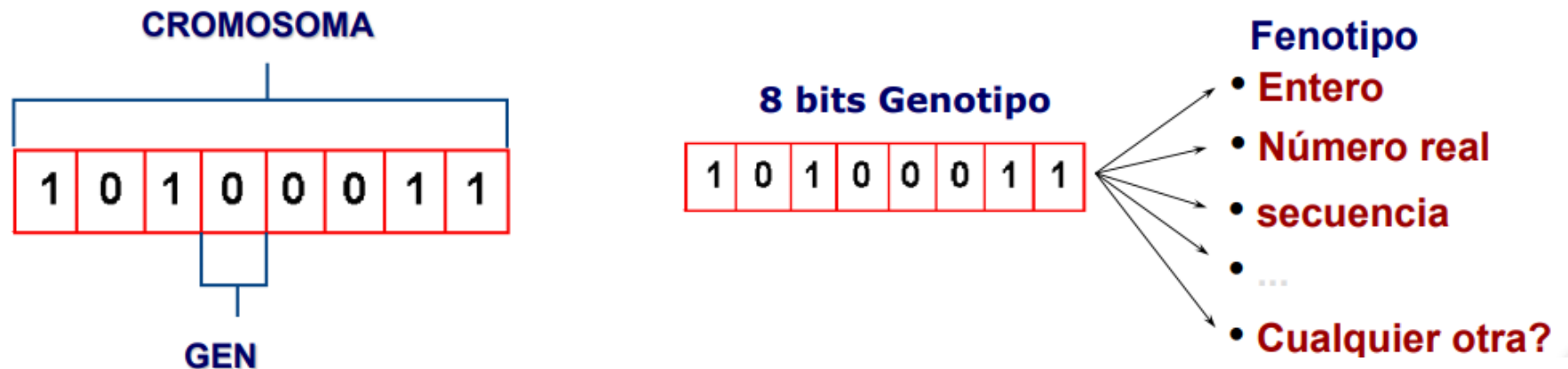
Representación

Debemos disponer de un mecanismo para codificar un individuo como un genotipo. Una vez elegida la representación, se evaluará a los genotipos (codificación) y los operadores genéticos a utilizar. Una codificación es discreta, en especial binaria.

Fenotipo: Es el conjunto de parámetros representando un cromosoma particular.

Genotipo: Es la totalidad de la información genética de un individuo.

Los individuos se representan como permutaciones.



Estrategias de Selección

Se debe de garantizar que los mejores individuos puedan ser padres (reproducirse) con una mayor posibilidad frente a los demás.

La presión selectiva determina en que grado la reproducción esta dirigida por los mejores individuos.

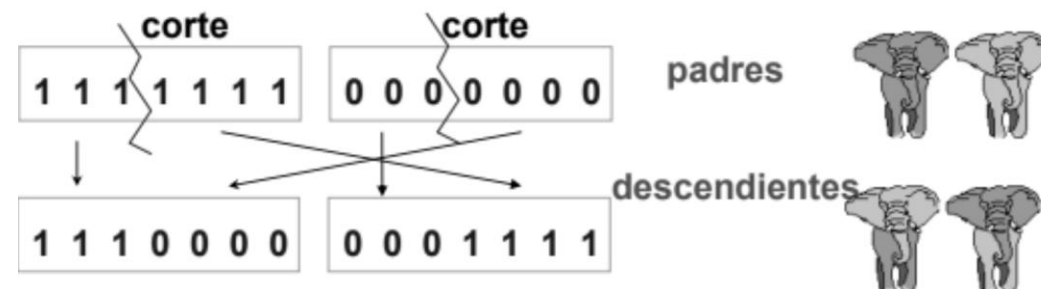
- **Selección por torneo (TS):** se escoge al individuo de mejor *Fitness* de entre Nts individuos seleccionados aleatoriamente.
- **Orden lineal (LR):** la población e orden en función de su fitness.
- **Selección aleatoria (RS).**
- **Emparejamiento Variado Inverso (NAM):** un padre escoge aleatoriamente a otro para emparejarse.
- **Selección por Ruleta:** se asigna una probabilidad de selección proporcional al valor del *fitness* del cromosoma. (Modelo clásico)

Operador de Cruce

Podríamos tener uno o más operadores de cruce para nuestra representación.

Algunos aspectos importantes a tener en cuenta son:

- Los hijos deberían heredar algunas características de cada padre. Si éste no es el caso, entonces estamos ante un operador de mutación.
- Se debe diseñar de acuerdo a la representación.
- La recombinación debe producir cromosomas válidos.
- Se utiliza con una probabilidad alta de actuación sobre cada pareja de padres a cruzar (entre 0.6 y 0.9), si no actúa los padres son los descendientes del proceso de recombinación de la pareja.

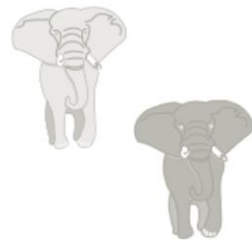


Operador de Mutación

Podemos tener uno o más operadores de mutación para nuestra representación.

Algunos aspectos importantes a tener en cuenta son:

- Debe permitir alcanzar cualquier parte del espacio de búsqueda.
- El tamaño de la mutación debe ser controlado.
- Debe producir cromosomas válidos.
- Se aplica con una probabilidad muy baja de actuación sobre cada descendiente obtenido tras aplicar el operador de cruce (incluidos los descendientes que coinciden con los padres porque el operador de cruce no actúa).



antes

1 1 1 1 1 1 1

después

1 1 1 0 1 1 1

GEN MUTADO

La mutación ocurre con una probabilidad para cada gen.

Estrategia de Reemplazo

La presión selectiva se ve también afectada por la forma en que los cromosomas de la población son reemplazados por los nuevos descendientes.

Podemos utilizar métodos de reemplazamiento aleatorios, o determinísticos. Podemos decidir **no reemplazar al mejor cromosoma de la población: Elitismo.**

Un modelo con alto grado de elitismo consiste en utilizar una población intermedia con todos los padres y todos los descendientes y seleccionar los mejores.

Reemplazar al peor de la población. Genera alta presión selectiva.

- **Torneo Restringido:** Se reemplaza al mas parecido de entre w ($w=3, \dots$). Mantiene una cierta diversidad.
- **Peor entre semejantes:** Se reemplaza el peor cromosoma del conjunto de los w ($w=3, \dots$) padres más parecidos al descendiente generado. Busca equilibrio entre diversidad y presión selectiva.
- Algoritmo de **Crowding Determinístico:** El hijo reemplaza a su padre más parecido. Mantiene diversidad.

Evaluation de un Individuo (*fitness*)

- Es el paso más costoso para la aplicación real.
- Puede ser una subrutina, un simulador, un proceso externo.
- Se puede usar funciones aproximadas para reducir el costo de evaluación.
- Cuando hay restricciones, éstas se pueden introducir en el costo de penalización.
- Con múltiples objetivos se busca una solución de compromiso.

Algoritmos genéticos

DESVENTAJAS Y LIMITACIONES

- Para problemas de alta complejidad la función de evaluación puede tornarse demasiado costosa en términos de tiempo y recursos.
- Puede haber casos en los cuales dependiendo los parámetros que se utilicen para la evaluación el algoritmo podría no llegar a converger en una solución óptima o bien terminar en una convergencia prematura con resultados no satisfactorios.
- No poseen una buena escalabilidad con la complejidad.
- La "mejor" solución lo es solo en comparación a otras soluciones por lo que no se tiene demasiado claro un criterio de cuándo detenerse.
- El diseño, la creación de la función de aptitud (*fitness*) y la selección de los criterios de mutación entre otros, necesitan conocimiento del problema para obtener buenos resultados.



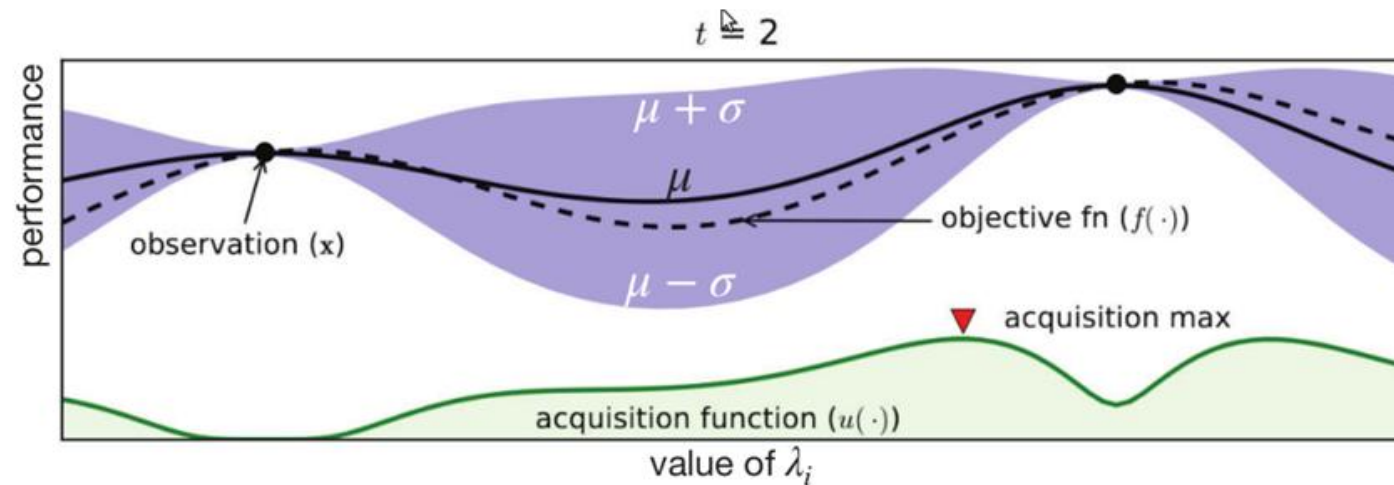
Bayesian Optimization

5. Aprendizaje automático

Bayes

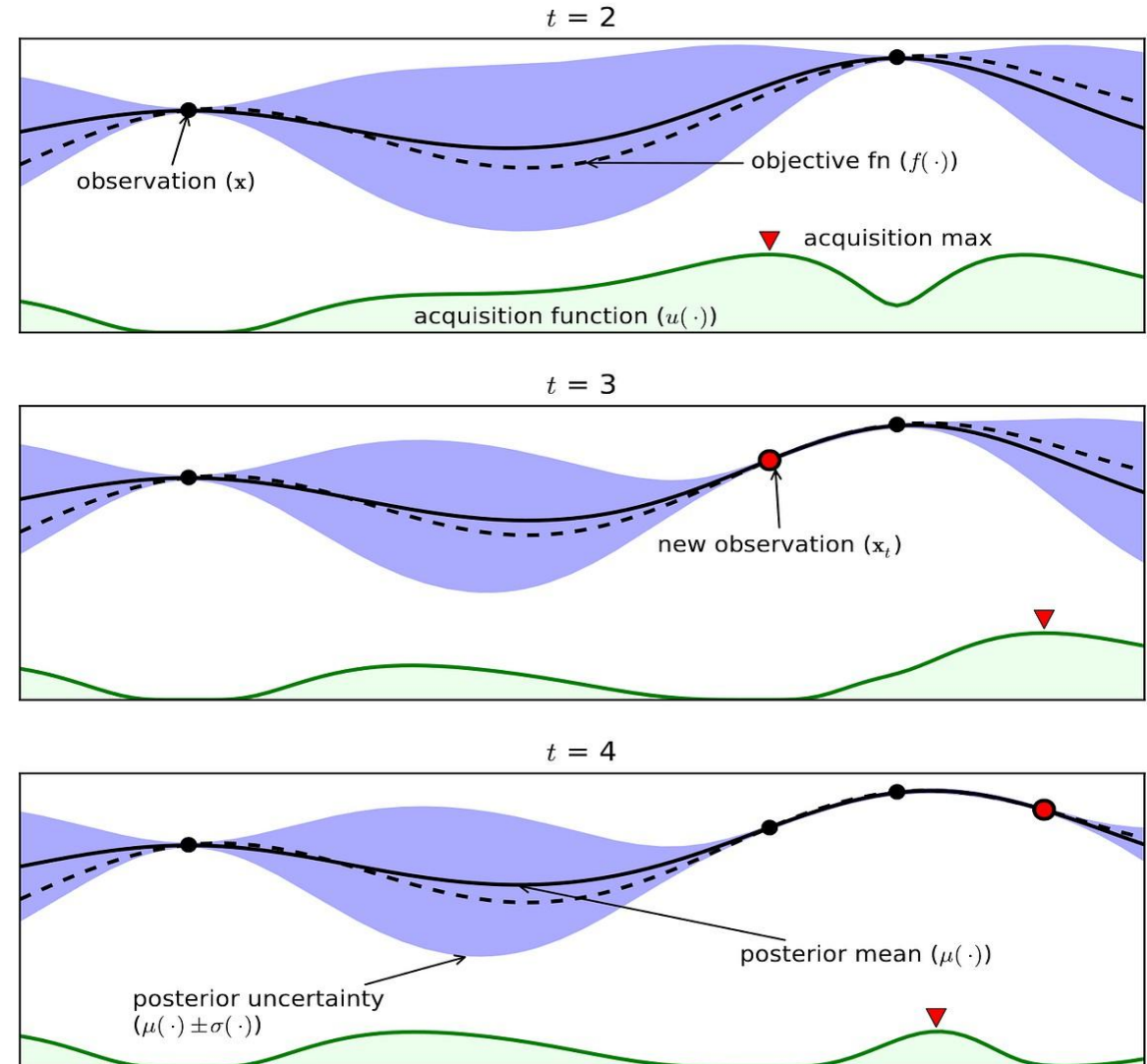
El método de Bayes lo que hace es modelar un proceso gaussiano a partir de un conjunto de muestras observables generando así una función muy próxima a la original, pero fácilmente extraíble e interpretable.

A partir de esta función aproximada, extrae la combinación de parámetros que maximizan el funcionamiento de nuestro algoritmo.



Bayes

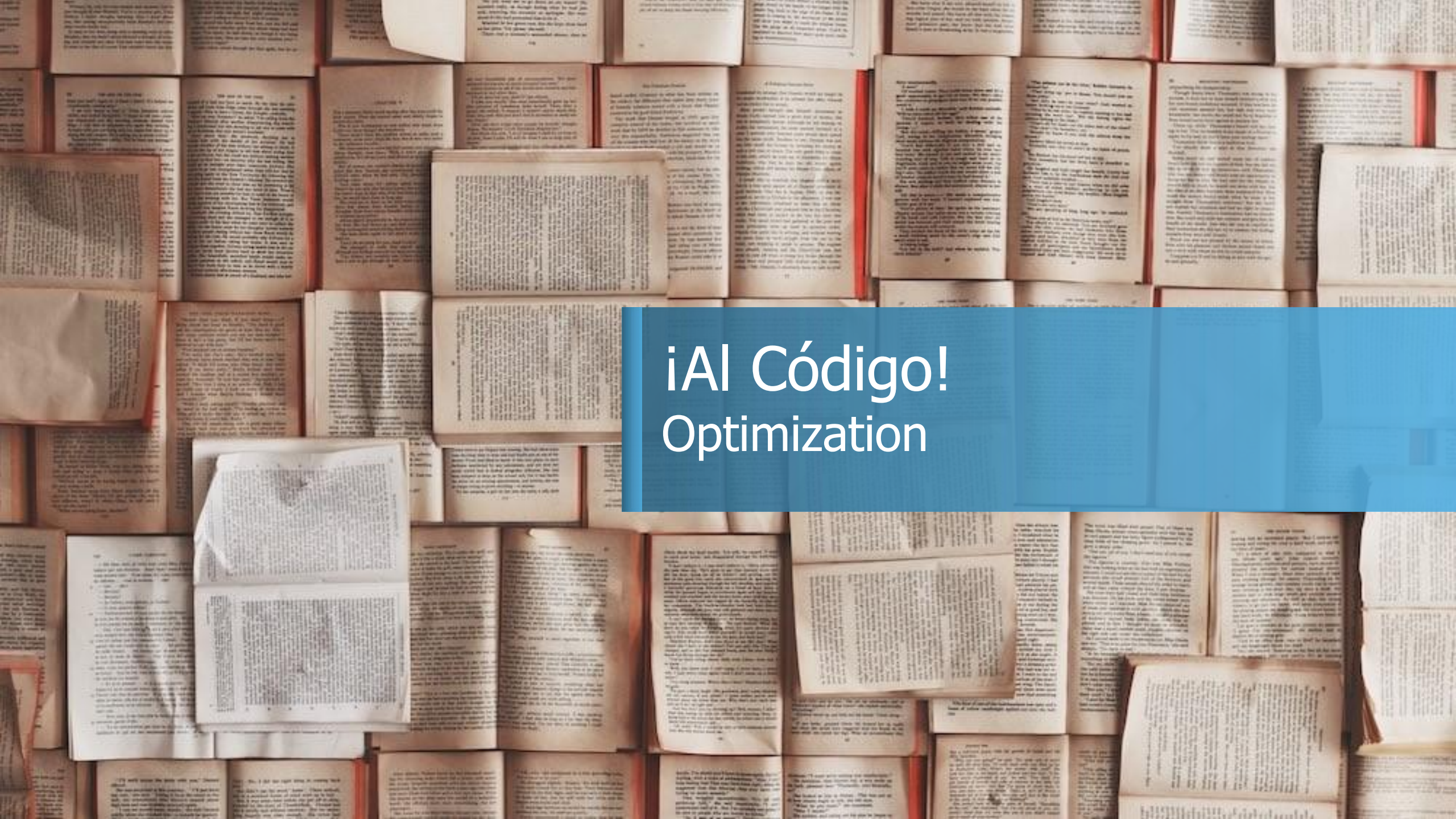
- Comenzar con algunas configuraciones de hiper parámetros (aleatorias).
- Ajustar un modelo sustituto (*surrogate*) para predecir el rendimiento de otras configuraciones.
- Una función de adquisición guía la propuesta de nuevos puntos a evaluar, equilibrando exploración y explotación.
- Seleccionar la mejor configuración según esa función.



Ventajas

The best of Both Worlds

- ~~• Alto consumo de recursos.~~
- ~~• Mala No muy buena escalabilidad.~~
- No garantiza encontrar un resultado óptimo, sólo el mejor de la combinación que se proporciona.
- Menos recursos que GridSearch al no comprobar todas las combinaciones.
- ~~• Dependemos de la aleatoriedad.~~
- ~~• No garantiza encontrar un resultado óptimo, pero si el mínimo global se encuentra dentro de los valores definidos, con > 60 iteraciones debería encontrar una combinación muy próxima.~~



iAl Código! Optimization

Tree-structured Parzen Estimator

Algoritmo de optimización bayesiana que selecciona hiperparámetros maximizando la probabilidad de mejora esperada, usado en *Optuna* y *Hyperopt*.

Enfoque probabilístico



- A diferencia de Gaussian Processes, este modela:

$$p(x | y) \text{ en lugar de } p(y | x)$$

- Esto permite trabajar mejor con:
 - Variables categóricas.
 - Espacios no continuos.
 - Estructuras jerárquicas (condicionales).

División del espacio:

- Se fija un umbral y^* (e.g., percentil $y = 0.1-0.2$)
- Se separan observaciones en: Buenas: $y < y^* \Rightarrow l(x)$, y Malas: $y \geq y^* \Rightarrow g(x)$

Tree-structured Parzen Estimator

Función de adquisición

- Se elige el siguiente punto maximizando:

$$EI(x) \propto \frac{l(x)}{g(x)}$$

- Favorece regiones donde:
 - Muchas configuraciones buenas.
 - Pocas configuraciones malas.
- Inicialización aleatoria (warm-up).
- KDE independientes por hiperparámetro.
- Soporte nativo para parámetros condicionales.

Ventajas y Limitaciones

- Más eficiente que Grid y Random Search.
- Escala bien a alta dimensionalidad.
- Muy flexible para pipelines ML reales.
- No modela correlaciones fuertes entre parámetros.
- Sensible a la elección de γ .
- Rendimiento depende del número de evaluaciones.

Sequential Model-based Algorithm Configuration

Algoritmo de optimización secuencial basada en modelos, diseñado para configurar algoritmos complejos (AutoML).

Modelo sustituto: es un Random Forest (RF) que predice el rendimiento de configuraciones de hiperparámetros para guiar la búsqueda sin entrenar todos los modelos.

- Usa RF en lugar de Gaussian Processes.
- Ventajas del RF:
 - Maneja variables mixtas (continuas, discretas, categóricas).
 - Robusto al ruido.
 - Escalable a muchos hiperparámetros.
- Predice:
 - Media del rendimiento.
 - Incertidumbre (a partir de los árboles).

Sequential Model-based Algorithm Configuration

Función de adquisición

- Usa Expected Improvement (EI):
$$EI(x) = E[\max(y^* - y(x), 0)]$$
- Balancea:
 - Exploración (incertidumbre alta).
 - Explotación (buen rendimiento esperado).

Proceso completo:

1. Muestreo inicial aleatorio
2. Entrenamiento del Random Forest
3. Optimización local de la adquisición
4. Evaluación real del modelo
5. Actualización iterativa

Ventajas y Limitaciones

- Muy robusto en problemas reales.
- Soporta restricciones y parámetros condicionales.
- Ideal para funciones objetivo caras.
- Mayor coste computacional.
- Más lento que TPE en problemas simples.
- Implementación compleja.



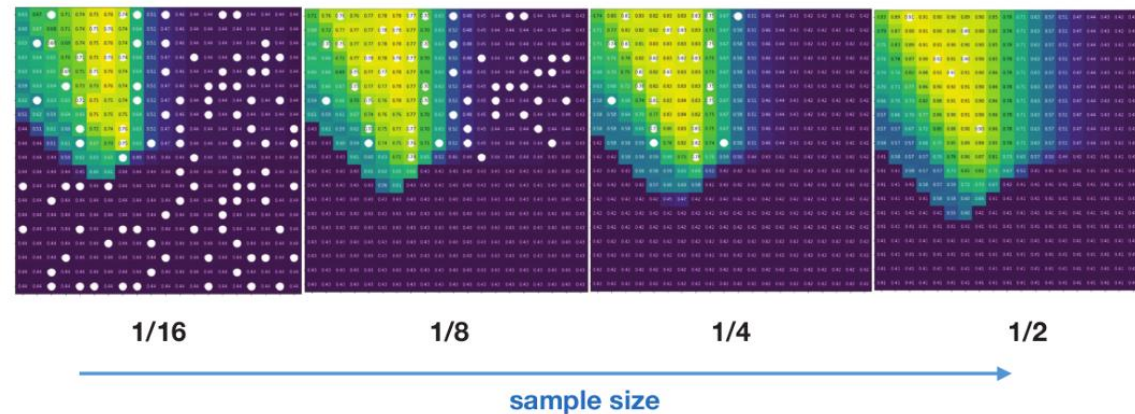
Multi-Fidelity Optimization

SHA

Idea principal: Este método intenta encontrar la mejor configuración de hiper-parámetros usando un presupuesto limitado.

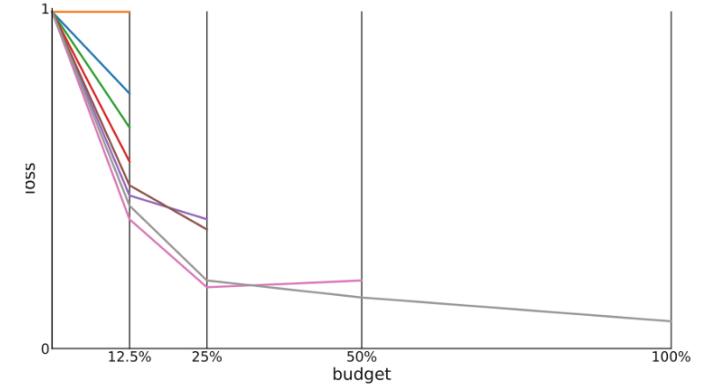
Pero hay un dilema: *¿cómo repartir ese presupuesto entre muchas configuraciones o pocas?*

- Entrenar con subconjuntos pequeños y determinar qué regiones podrían ser interesantes para evaluar en mayor profundidad.
- Muestrear aleatoriamente candidatos y evaluarlos con un pequeño subconjunto de datos.
- Por ejemplo, volver a entrenar el 50 % de los mejores candidatos con el doble de datos.



Algoritmo

- SHA aplicado a ocho algoritmos/configuraciones.
- Después de evaluar todos los algoritmos con 1/8 del presupuesto total, se descarta la mitad, y el presupuesto de los algoritmos restantes se duplica.



SUCCESSIVEHALVING (Finite horizon)

input: Budget B , and n arms where $\ell_{i,k}$ denotes the k th loss from the i th arm, maximum size R , $\eta \geq 2$ ($\eta = 3$ by default).

Initialize: $S_0 = [n]$, $s = \min\{t \in \mathbb{N} : nR(t+1)\eta^{-t} \leq B, t \leq \log_\eta(\min\{R, n\})\}$.

For $k = 0, 1, \dots, s$

Set $n_k = \lfloor n\eta^{-k} \rfloor$, $r_k = \lfloor R\eta^{k-s} \rfloor$

Pull each arm in S_k for r_k times.

Keep the best $\lfloor n\eta^{-(k+1)} \rfloor$ arms in terms of the r_k th observed loss as S_{k+1} .

Output : $\hat{i}, \ell_{\hat{i},R}$ where $\hat{i} = \arg \min_{i \in S_{s+1}} \ell_{i,R}$

Limitaciones

- El método SHA sufre del compromiso entre presupuesto y número de configuraciones.
- Dado un presupuesto total, el usuario debe decidir de antemano si: 1) probar muchas configuraciones y asignarles un presupuesto pequeño a cada una, o 2) probar pocas configuraciones y asignarles un presupuesto mayor.
- Asignar un presupuesto demasiado pequeño puede resultar en terminar prematuramente configuraciones que son buenas.
- Asignar un presupuesto demasiado grande puede resultar en ejecutar configuraciones malas durante demasiado tiempo, desperdiciando recursos.

Visualización mental:

- Imagina que tienes \$100 para repartir entre diferentes candidatos.
 - Si das \$10 a 10 candidatos → muchas opciones exploradas, pero no se desarrolla ninguna completamente.
 - Si das \$50 a 2 candidatos → cada uno se desarrolla bien, pero podrías estar ignorando otras opciones potencialmente mejores.

HyperBand

Idea principal: HyperBand soluciona el problema de SHA cuando se seleccionan configuraciones de manera aleatoria, equilibrando el número de configuraciones y el presupuesto asignado a cada una.

- Divide el presupuesto total en varias combinaciones de:
 - Número de configuraciones a probar.
 - Presupuesto asignado a cada configuración.
- Para cada combinación, ejecuta SHA sobre un subconjunto aleatorio de configuraciones.
- Esto permite explorar muchas configuraciones rápidamente y al mismo tiempo dar más recursos a las mejores, evitando el trade-off extremo de SHA tradicional.

Ventaja clave:

HyperBand es más eficiente que SHA, porque no tienes que decidir de antemano si probar muchas configuraciones con poco presupuesto o pocas con mucho presupuesto: lo hace automáticamente probando varias combinaciones.

Algoritmo

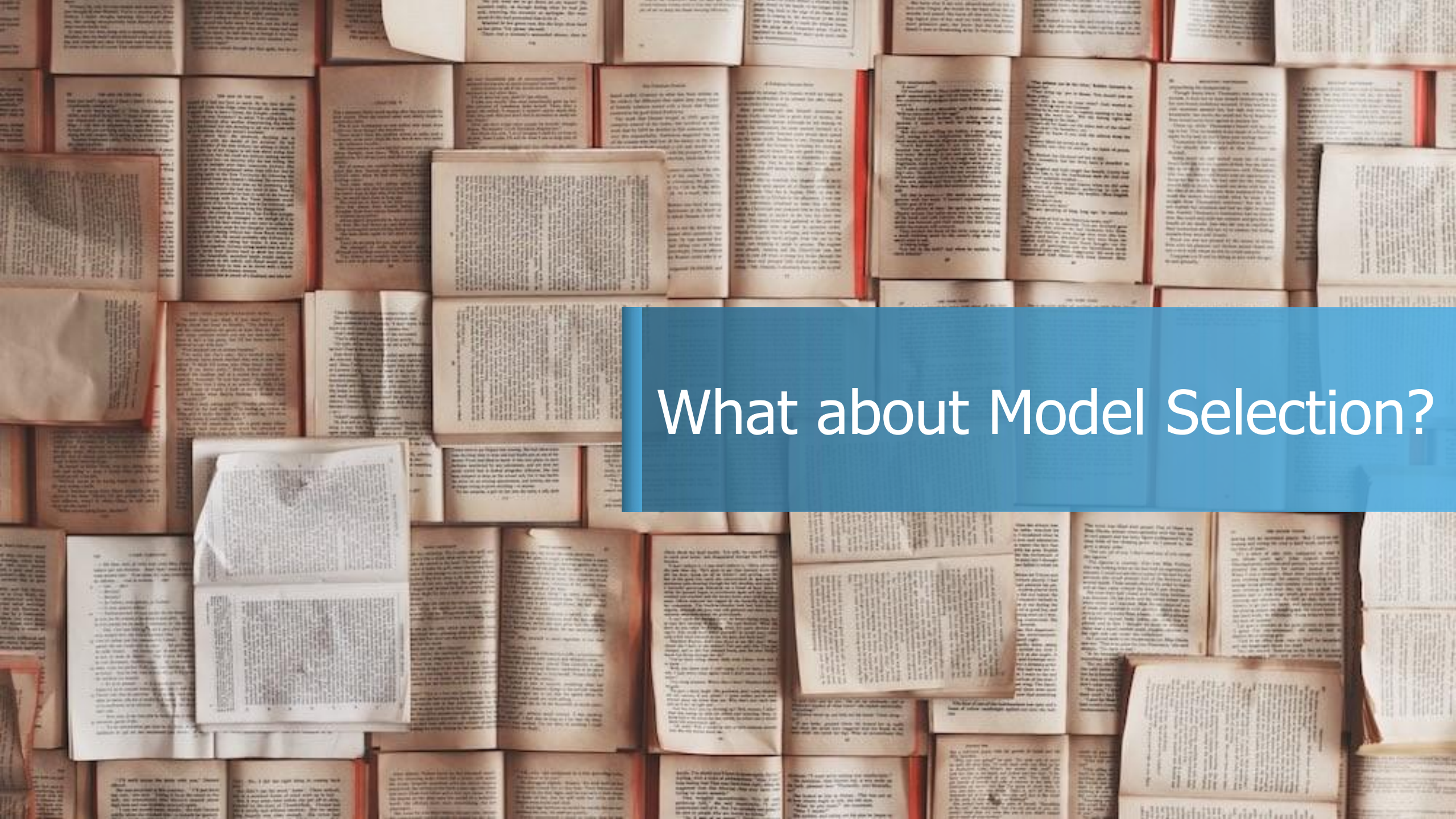
Algorithm 1: HYPERBAND algorithm for hyperparameter optimization.

```

input          :  $R, \eta$  (default  $\eta = 3$ )
initialization :  $s_{\max} = \lfloor \log_{\eta}(R) \rfloor, B = (s_{\max} + 1)R$ 
1 for  $s \in \{s_{\max}, s_{\max} - 1, \dots, 0\}$  do
2    $n = \lceil \frac{B}{R} \frac{\eta^s}{(s+1)} \rceil, \quad r = R\eta^{-s}$ 
   // begin SUCCESSIVEHALVING with  $(n, r)$  inner loop
3    $T = \text{get\_hyperparameter\_configuration}(n)$ 
4   for  $i \in \{0, \dots, s\}$  do
5      $n_i = \lfloor n\eta^{-i} \rfloor$ 
6      $r_i = r\eta^i$ 
7      $L = \{\text{run\_then\_return\_val\_loss}(t, r_i) : t \in T\}$ 
8      $T = \text{top\_k}(T, L, \lfloor n_i/\eta \rfloor)$ 
9   end
10 end
11 return Configuration with the smallest intermediate loss seen so far.

```

- El bucle interno ejecuta SHA para valores fijos de n y r .
- El bucle externo itera sobre diferentes valores de n y r .

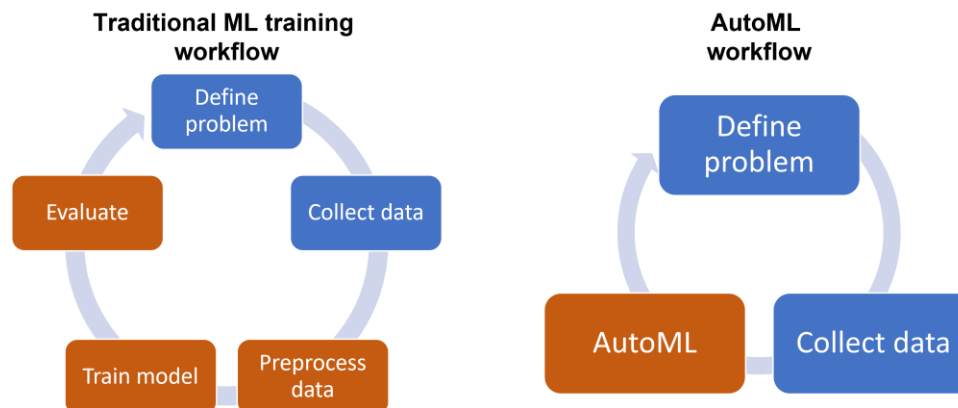


What about Model Selection?

AutoML

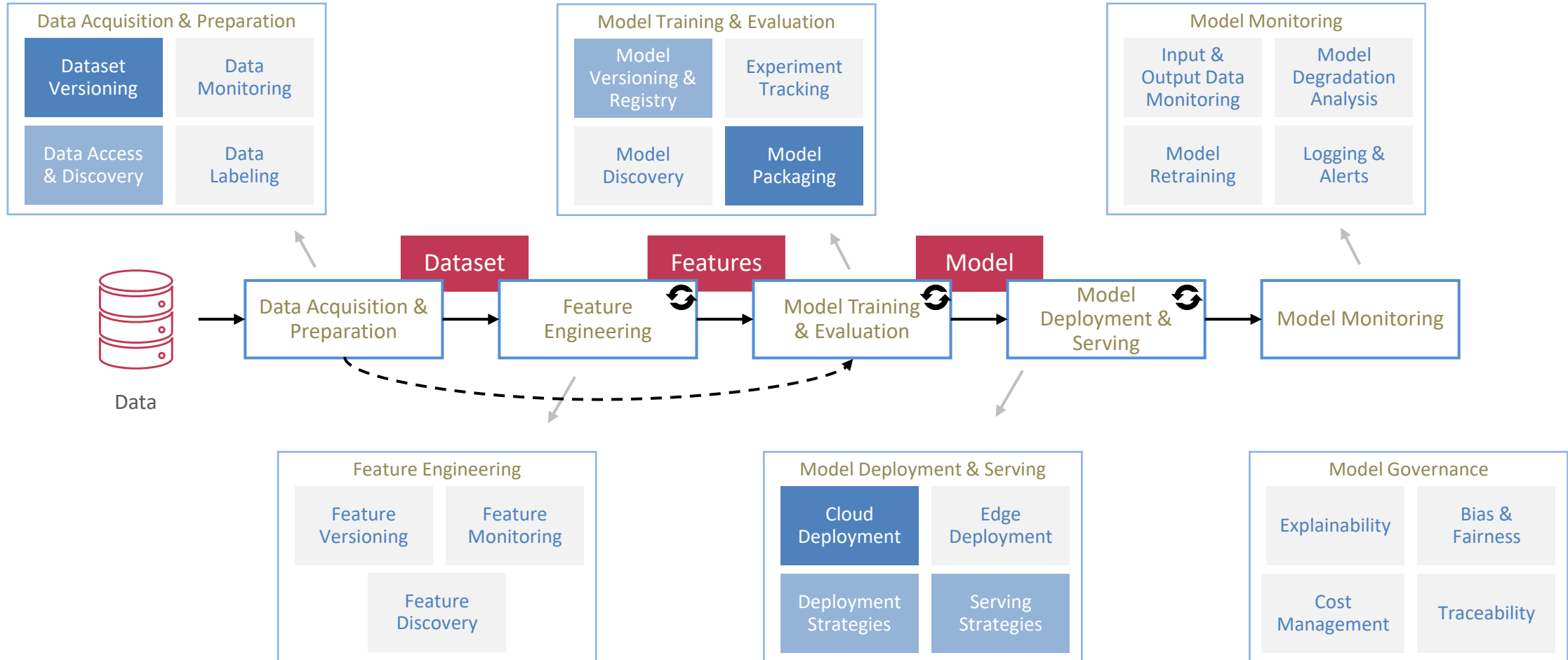
AutoML automatiza tareas complejas del aprendizaje automático.

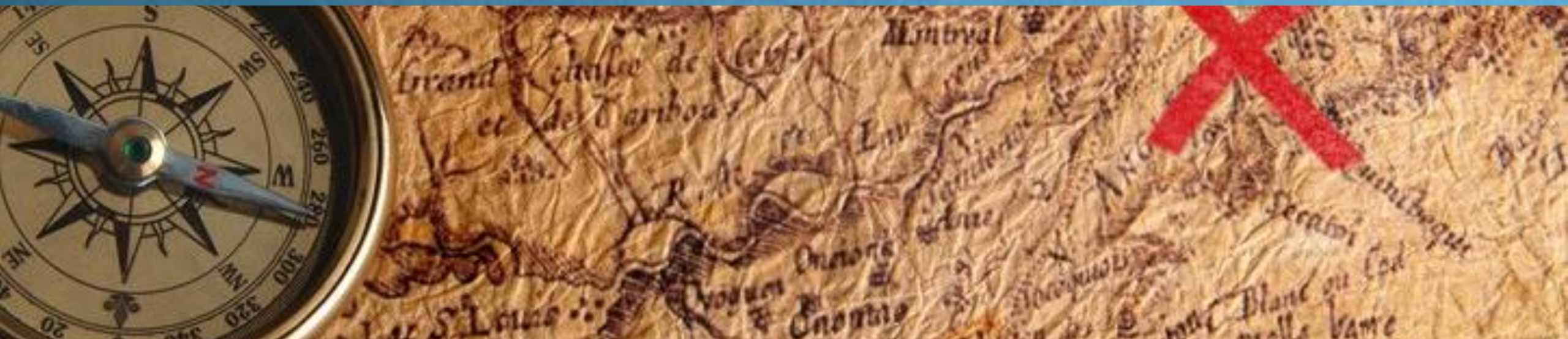
- Ahorra tiempo y esfuerzo al optimizar procesos manuales de un DS.
- Facilita el acceso al aprendizaje automático para no expertos.
- Identifica el modelo más adecuado mediante selección de algoritmos.
- Emplea técnicas avanzadas para el ajuste de hiperparámetros.
- Herramientas como Auto-Sklearn y Google AutoML facilitan el uso.
- **Se puede usar para identificar modelos exploradores según el problema.**



MLOps

--> Shortcut ↻ Iterative process
Feedback loops omitted for simplicity.





Jordina Torrents - jordina.torrents@ext.salle.url.edu